**Exam. Seat No.** |  |

**Walwa Taluka Education Society's**

# YASHWANTRAO CHAVAN MAHAVIDYALAYA, ISLAMPUR

**Tal: Walwa, Dist: Sangli.**

## Department Of Computer Science

## LABORATORY CERTIFICATE

This is to certify that, Mr./Miss. _____

Roll No. _____ has satisfactorily completed the course of Experiment in **Major Practical V (a) Web Development using Python Lab** as per syllabus laid down by Shivaji University for B.Sc. **Computer Science (Optional) Part –III Sem V** during the academic year 2026-2027.

**Date: / / 2026**

**Teacher in charge** **Examiner** **Head**
**Department of Computer Science**

# INDEX

## Major Practical V (a) Web Development using Python Lab

| Sr. No. | Title | Page No. | Date | Remark |
|---|---|---|---|---|
| 1 | Write Python programs to demonstrate variables, data types, and operators | | 06-07-2026 | |
| 2 | Implement control statements (if-else, for loop, while loop) with practical examples | | 20-07-2026 | |
| 3 | Create user-defined functions and demonstrate parameter passing | | 03-08-2026 | |
| 4 | Perform operations on lists, tuples, dictionaries, and sets | | 17-08-2026 | |
| 5 | Implement exception handling for common errors (ZeroDivisionError, FileNotFoundError) | | 24-08-2026 | |
| 6 | Create programs for file reading and writing operations | | 07-09-2026 | |
| 7 | Set up virtual environment and install Django using pip | | 21-09-2026 | |
| 8 | Install Django and create a new Django project | | 28-09-2026 | |
| 9 | Create a Django app and configure it in the project settings | | 05-10-2026 | |
| 10 | Design HTML pages using Django templates with variables and template tags | | 05-10-2026 | |
| 11 | Design HTML pages using Django templates with variables and template tags. | | 05-10-2026 | |

**Experiment 1: Write Python programs to demonstrate variables, data types, and operators**

### 1.1 Python Program to Demonstrate Variables

**Problem Analysis:**

The purpose of this program is to demonstrate the use of variables in Python. Variables are used to store data values in memory. In this program, different values such as name, age, and marks are assigned to variables and then displayed on the screen.

**Program**

```
name = "Sujit"
age = 20
marks = 85.5
print("Name:", name)
print("Age:", age)
print("Marks:", marks)
```

**Output**

```
Name: Rahul
Age: 20
Marks: 85.5
```

### 1.2 Python Program to Demonstrate Data Types

**Problem Analysis:**

The purpose of this program is to demonstrate different built-in data types available in Python. The program creates variables containing integer, floating-point, string, and Boolean values. The type() function is used to display the data type of each variable.

**Program**

```
a = 10
b = 10.5
c = "Python"
d = True
print(type(a))
print(type(b))
print(type(c))
print(type(d))
```

**Output:**

```
<class 'int'>
<class 'float'>
<class 'str'>
<class 'bool'>
```

### 1.3 Python Program to Demonstrate Operators

**Problem Analysis:**

The purpose of this program is to demonstrate arithmetic operators in Python. The program accepts two numbers from the user and performs basic arithmetic operations such as addition, subtraction, multiplication, division, and modulus. The results of all operations are displayed.

**Program**

```
a = int(input("Enter first number: "))
b = int(input("Enter second number: "))
print("Addition:", a + b)
print("Subtraction:", a - b)
print("Multiplication:", a * b)
print("Division:", a / b)
print("Modulus:", a % b)
```

**Output**

```
Enter first number: 21
Enter second number: 32
Addition: 53
Subtraction: -11
Multiplication: 672
Division: 0.65625
Modulus: 21
```

**Experiment 2: Implement control statements (if-else, for loop, while loop) with practical examples**

### 2.1 Python Program to Demonstrate if-else Statement

**Problem Analysis**

The purpose of this program is to demonstrate the use of the if-else control statement in Python. The program accepts a number from the user and checks whether the number is positive or negative. If the number is greater than or equal to zero, it displays “Positive Number”; otherwise, it displays “Negative Number”.

**Program**

```
num = int(input("Enter a number: "))
if num >= 0:
   print("Positive Number")
else:
   print("Negative Number")
```

**Output:**

```
Enter a number: 25
Positive Number

Enter a number: -45
Negative Number
```

### 2.2 Python Program to Demonstrate for Loop

**Problem Analysis**

The purpose of this program is to demonstrate the use of a for loop in Python. The program uses the for loop to repeat a set of statements for a fixed number of times. It displays numbers from 1 to 10, showing how a loop can be used to perform repetitive tasks efficiently.

**Program**

```
for i in range(1, 11):
   print(i)
```

**Output:**

```
1
2
3
4
5
6
7
8
9
10
```

## 2.3 Python Program to Demonstrate while Loop

### Problem Analysis

The purpose of this program is to demonstrate the use of a while loop in Python. The program starts with a counter value of 1 and repeatedly executes the loop as long as the counter is less than or equal to 20. After displaying each number, the counter is increased by one.

### Program

```
i = 1
while i <= 20:
    print(i)
    i = i + 1
```

### Output:

```
1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
```

**Experiment 3: Create User-Defined Functions and Parameter Passing in Python**

### 3.1 Python Program to Demonstrate a User-Defined Function

**Problem Analysis**

The purpose of this program is to demonstrate how to create and use a user-defined function in Python. A function named greet() is defined to display a simple greeting message. The function is then called from the main program to execute the statements inside it.

**Program**

```
def welcome():
    print("Welcome to Python Programming")
```

**Output:**

```
welcome()
Welcome to Python Programming
```

### 3.2 Python Program to Demonstrate Function with Parameters

**Problem Analysis**

The purpose of this program is to demonstrate parameter passing to a user-defined function. The function add() accepts two parameters and calculates their sum. The values are provided by the user and passed to the function when it is called.

**Program**

```
def add(a, b):
    print("Sum =", a + b)
x = int(input("Enter first number: "))
y = int(input("Enter second number: "))
add(x, y)
```

**Output:**

```
Enter first number: 24
Enter second number: 43
Sum = 67
```

### 3.3 Python Program to Demonstrate Function with Return Value

**Problem Analysis**

The purpose of this program is to demonstrate a function that accepts parameters and returns a value. The function square() accepts a number as a parameter, calculates its square, and returns the result to the calling program.

**Program**

```
def square(n):
    return n * n
```

```
num = int(input("Enter a number: "))
result = square(num)
print("Square =", result)
```

**Output:**

```
Enter a number: 43
Square = 1849
```

### 3.4 Python Program to Demonstrate Default Parameter

**Problem Analysis**

The purpose of this program is to demonstrate the use of a default parameter in a user-defined function. The function display() has a default value for the name parameter. If no argument is passed, the default value is used; otherwise, the supplied value is displayed.

**Program**

```
def display(name="Student"):
    print("Hello", name)
display()
display("Rahul")
```

**Output:**

```
Hello Student
Hello Rahul
```

### 3.5 Python Program to Demonstrate Multiple Parameters

**Problem Analysis**

The purpose of this program is to demonstrate passing multiple parameters to a user-defined function. The function student() accepts the student's name, roll number, and marks as parameters and displays all the information.

**Program**

```
def student(name, roll, marks):
    print("Name:", name)
    print("Roll No:", roll)
    print("Marks:", marks)

student("Amit", 101, 85)
```

**Output:**

```
Name: Amit
Roll No: 101
Marks: 85
```

**Experiment 4: Python Programs on Lists, Tuples, Dictionaries and Sets**

**4.1 Program to Perform Operations on a List**

**Problem Analysis**

The purpose of this program is to demonstrate basic operations on a Python list. The program creates a list of numbers and performs operations such as adding an element, removing an element, and displaying the list. Lists are ordered and mutable collections in Python.

**Program**

```
numbers = [10, 20, 30, 40]
print("Original List:", numbers)
numbers.append(50)
print("After Append:", numbers)
numbers.remove(20)
print("After Remove:", numbers)
numbers.sort()
print("Sorted List:", numbers)
```

**Output**

```
Original List: [10, 20, 30, 40]
After Append: [10, 20, 30, 40, 50]
After Remove: [10, 30, 40, 50]
Sorted List: [10, 30, 40, 50]
```

**4.2 Program to Perform Operations on a Tuple**

**Problem Analysis**

The purpose of this program is to demonstrate basic operations on a Python tuple. The program creates a tuple containing numbers and performs operations such as accessing an element, finding the length, and counting an element. A tuple is an ordered and immutable collection.

**Program**

```
numbers = (10, 20, 30, 20, 40)
print("Tuple:", numbers)
print("First Element:", numbers[0])
print("Length:", len(numbers))
print("Count of 20:", numbers.count(20))
```

**Output:**

```
Tuple: (10, 20, 30, 20, 40)
First Element: 10
Length: 5
Count of 20: 2
```

### 4.3 Program to Perform Operations on a Dictionary

**Problem Analysis:**

The purpose of this program is to demonstrate basic operations on a Python dictionary. The program creates a dictionary containing student information using key-value pairs. It displays the dictionary, adds a new item, updates an existing value, and accesses a particular value using its key.

**Program**

```
student = {
   "name": "Rahul",
   "roll": 101,
   "marks": 85
}
print("Original Dictionary:", student)
student["city"] = "Pune"
student["marks"] = 90
print("Updated Dictionary:", student)
print("Student Name:", student["name"])
```

**Output**

```
Original Dictionary: {'name': 'Rahul', 'roll': 101, 'marks': 85}
Updated Dictionary: {'name': 'Rahul', 'roll': 101, 'marks': 90, 'city': 'Pune'}
Student Name: Rahul
```

### 4.4 Program to Perform Operations on a Set

**Problem Analysis:**

The purpose of this program is to demonstrate basic operations on Python sets. The program creates two sets and performs union, intersection, and difference operations. Sets are unordered collections that do not allow duplicate elements.

**Program**

```
A = {10, 20, 30, 40}
B = {30, 40, 50, 60}
print("Set A:", A)
print("Set B:", B)
print("Union:", A | B)
print("Intersection:", A & B)
print("Difference:", A - B)
```

**Output**

```
Set A: {40, 10, 20, 30}
Set B: {40, 50, 60, 30}
Union: {40, 10, 50, 20, 60, 30}
Intersection: {40, 30}
Difference: {10, 20}
```

**Experiment 5: Python Programs to Demonstrate Exception Handling**

**5.1Program to Handle ZeroDivisionError**

**Problem Analysis:**

The purpose of this program is to demonstrate exception handling for ZeroDivisionError. Division by zero is not allowed in Python and causes an exception. The try block contains the division operation, while the except block catches the error and displays a suitable message instead of terminating the program.

**Program**

```
try:
    a = int(input("Enter first number: "))
    b = int(input("Enter second number: "))
    result = a / b
    print("Result:", result)
except ZeroDivisionError:
    print("Error: Cannot divide by zero.")
```

**Output**

```
Enter first number: 15
Enter second number: 3
Result: 5.0

Enter first number: 5
Enter second number: 0
Error: Cannot divide by zero.
```

**5.2 Program to Handle FileNotFoundError**

**Problem Analysis:**

The purpose of this program is to demonstrate exception handling for FileNotFoundError. This error occurs when a program tries to open a file that does not exist. The try block attempts to open and read the file, while the except block handles the error and displays an appropriate message.

**Program**

```
try:
    file = open("data.txt", "r")
    print(file.read())
    file.close()
except FileNotFoundError:
    print("Error: File not found.")
```

**Output**

```
Error: File not found.
```

## Experiment 6: Python Programs for File Reading and Writing Operations

### 6.1 Program to Write Data into a File

**Problem Analysis:**

The purpose of this program is to demonstrate how to write data into a text file using Python. The open() function is used with write mode ("w"), which creates a new file or overwrites an existing file. The write() function is then used to store text in the file.

**Program**

```
file = open("data.txt", "w")
file.write("Welcome to Python Programming")
file.close()
print("Data written successfully.")
```

**Output**

Data written successfully.

### 6.2 Program to Read Data from a File

**Problem Analysis:**

The purpose of this program is to demonstrate how to read data from a text file using Python. The file is opened in read mode ("r"), and the read() function is used to read the complete contents of the file. Finally, the file is closed after reading.

**Program**

```
file = open("data.txt", "r")
data = file.read()
print("File Content:")
print(data)
file.close()
```

**Output**

File Content:
Welcome to Python Programming

### 6.3 Program to Append Data to a File

**Problem Analysis:**

The purpose of this program is to demonstrate how to add new data to an existing file without deleting its previous contents. The file is opened using append mode ("a"), and the write() function adds new text at the end of the file.

**Program**

```
file = open("data.txt", "a")
file.write("\nThis is a File Handling Program.")
file.close()
print("Data appended successfully.")
```

**Output:**

Data appended successfully.

### 6.4 Program to Read a File Line by Line

**Problem Analysis:**

The purpose of this program is to demonstrate reading a text file line by line. The file is opened in read mode, and a for loop is used to read and display each line separately. This method is useful when working with files containing multiple lines of data.

**Program**

```
file = open("data.txt", "r")
for line in file:
    print(line, end="")
file.close()
```

**Output**

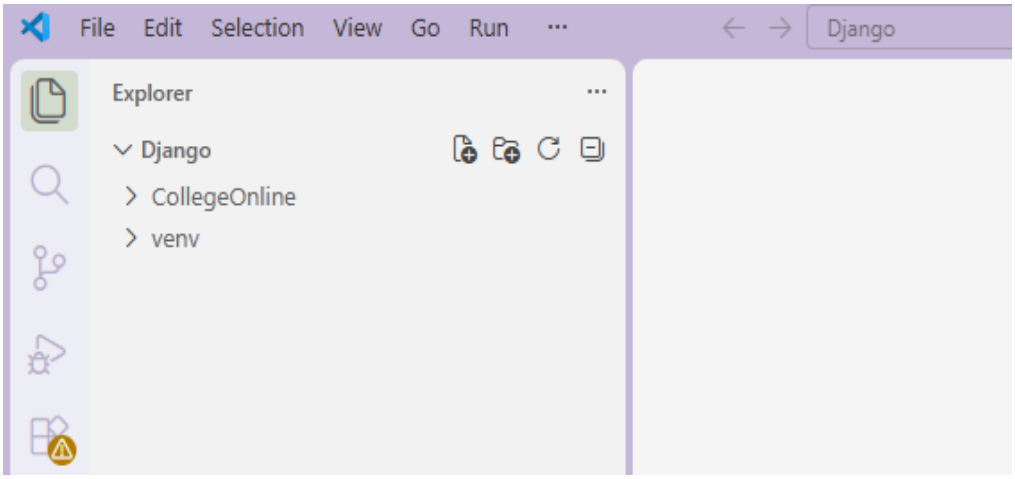
Welcome to Python Programming
This is a File Handling Program.

**Experiment 7: Set up virtual environment and install Django using pip**

```
PS D:\Python web Practicals for journal\Django> python -m venv venv
PS D:\Python web Practicals for journal\Django> venv/Scripts/Activate
(venv) PS D:\Python web Practicals for journal\Django>
Step 2. Install django using pip
(venv) PS D:\Python web Practicals for journal\Django> pip install django
Collecting django
 Using cached django-6.1.1-py3-none-any.whl.metadata (3.9 kB)
Collecting asgiref>=3.9.1 (from django)
 Using cached asgiref-3.12.1-py3-none-any.whl.metadata (9.4 kB)
Collecting sqlparse>=0.5.0 (from django)
 Using cached sqlparse-0.6.0-py3-none-any.whl.metadata (6.0 kB)
Collecting tzdata (from django)
Using cached tzdata-2026.4-py2.py3-none-any.whl.metadata (1.4 kB)
Using cached django-6.1.1-py3-none-any.whl (8.4 MB)
Using cached asgiref-3.12.1-py3-none-any.whl (25 kB)
Using cached sqlparse-0.6.0-py3-none-any.whl (50 kB)
Using cached tzdata-2026.4-py2.py3-none-any.whl (347 kB)
Installing collected packages: tzdata, sqlparse, asgiref, django
Successfully installed asgiref-3.12.1 django-6.1.1 sqlparse-0.6.0 tzdata-2026.4
[notice] A new release of pip is available: 24.3.1 -> 26.2.1
[notice] To update, run: python.exe -m pip install --upgrade pip
(venv) PS D:\Python web Practicals for journal\Django>
```

**Experiment 8: Install Django and create a new Django project**

(venv) PS D:\Python web Practicals for journal\Django> django-admin startproject CollegeOnline
(venv) PS D:\Python web Practicals for journal\Django> cd CollegeOnline
(venv) PS D:\Python web Practicals for journal\Django\CollegeOnline>



**Experiment 9: Create a Django app and configure it in the project settings**

Step 1: First run your project with welcome page as output

(venv) PS D:\Python web Practicals for journal\Django\CollegeOnline> python manage.py runserver

Watching for file changes with StatReloader

Performing system checks...

System check identified no issues (0 silenced).

You have 18 unapplied migration(s). Your project may not work properly until you apply the migrations for app(s): admin, auth, contenttypes, sessions.

Run 'python manage.py migrate' to apply them

Django version 6.1.1, using settings 'CollegeOnline.settings'

Starting WSGI development server at http://127.0.0.1:8000/

Quit the server with CTRL-BREAK

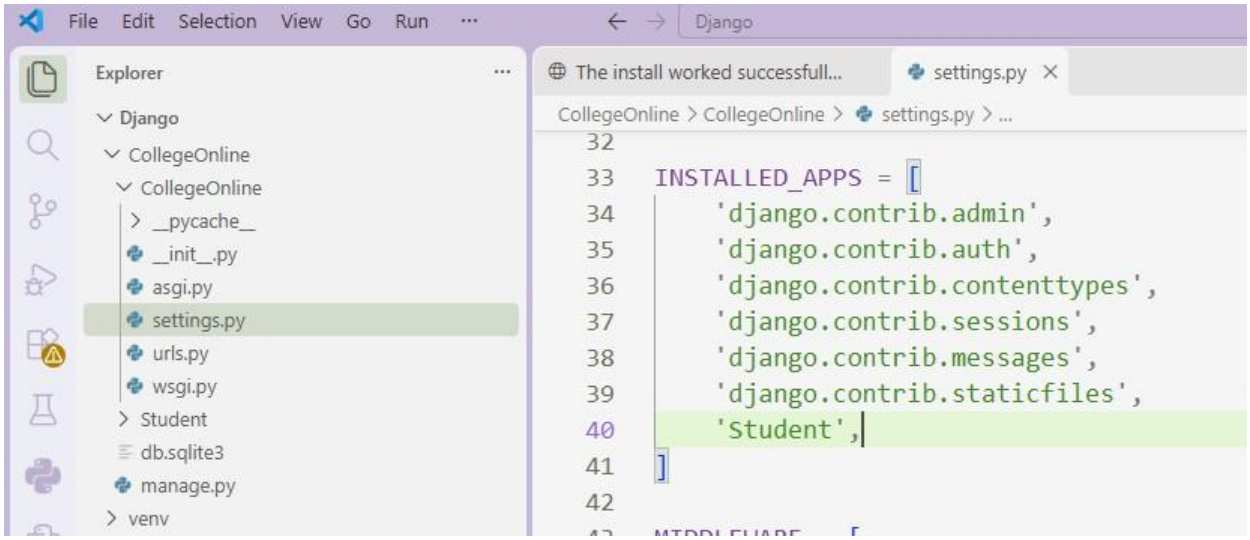



Step 2: Stop server using Ctrl+c and create app and configure it in the project settings

(venv) PS D:\Python web Practicals for journal\Django\CollegeOnline> python manage.py startapp Student

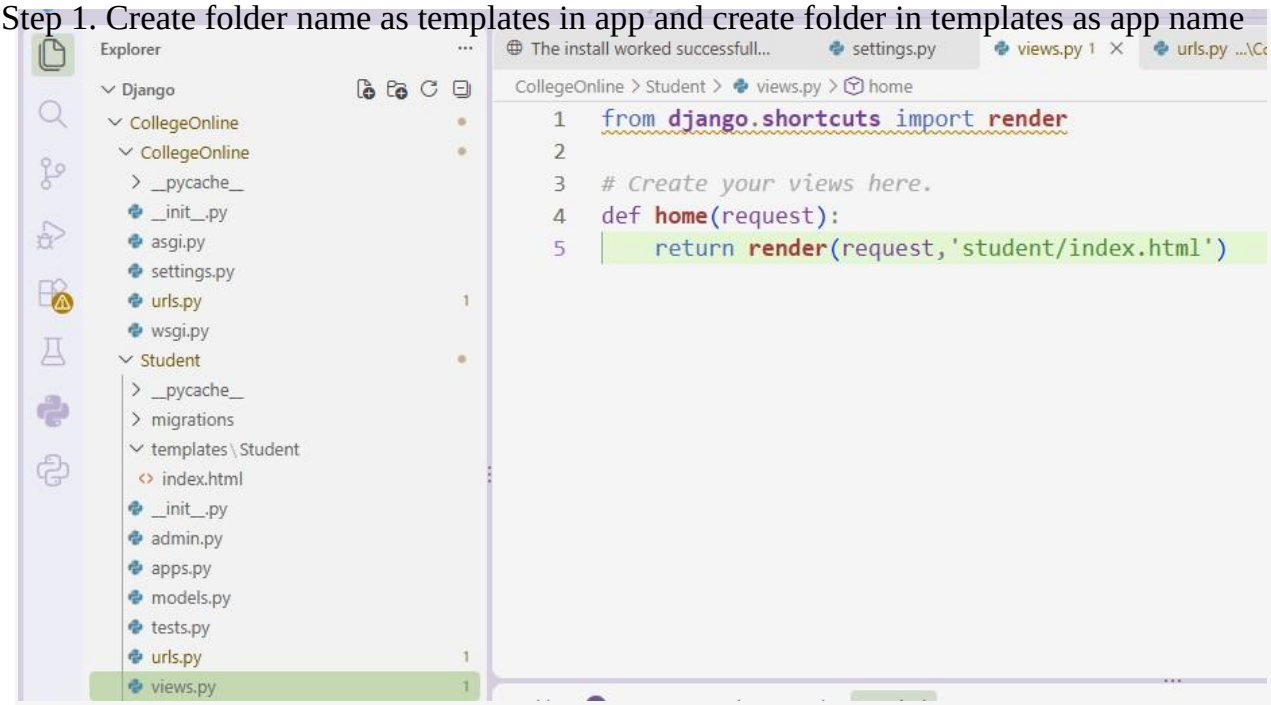
(venv) PS D:\Python web Practicals for journal\Django\CollegeOnline>

Step 3: Go to settings.py file and add app



**Experiment 10: Create simple views and map URLs to display "Hello World" page**

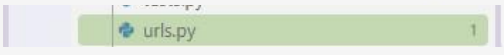
Step 1. Create folder name as templates in app and create folder in templates as app name

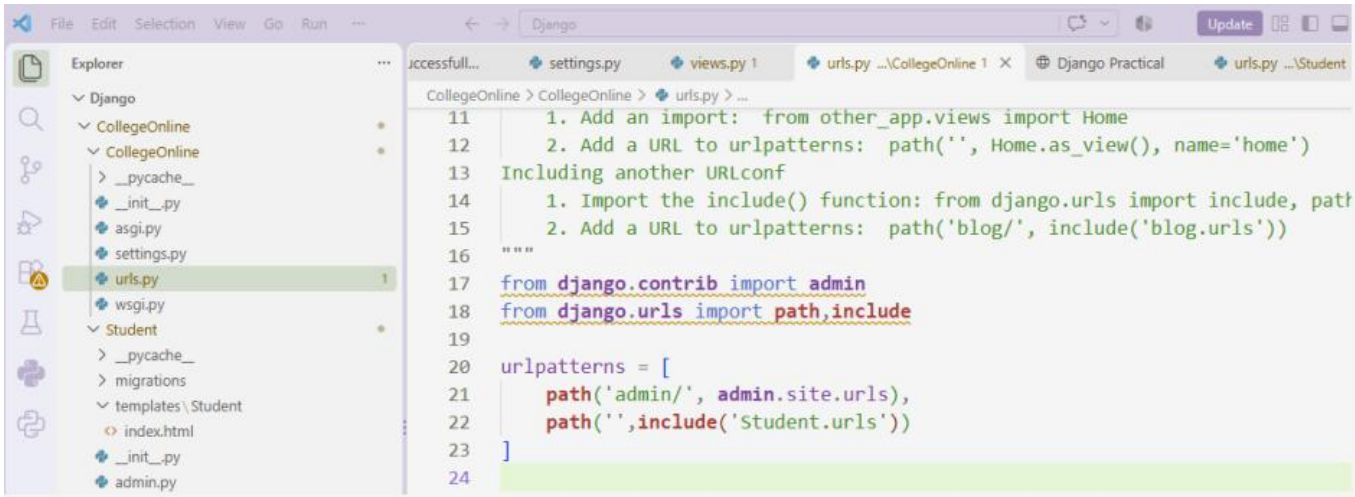



**Step 2. Add templates folder in settings.py in TEMPLATES = [] list**




**Step 3. Write definition in app_name/views.py**






**Step 6. Run the project**

(venv) PS D:\Python web Practicals for journal\Django\CollegeOnline> python manage.py runserver

Watching for file changes with StatReloader

Performing system checks...

System check identified no issues (0 silenced).

You have 18 unapplied migration(s). Your project may not work properly until you apply the migrations for app(s): admin, auth, contenttypes, sessions.

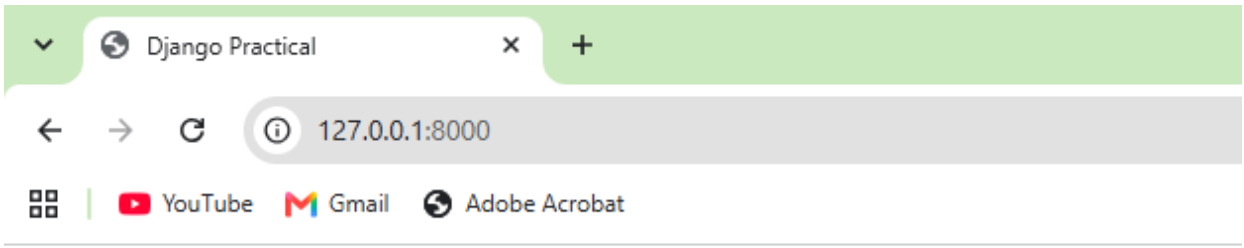
Run 'python manage.py migrate' to apply them.

September 26, 2026 - 09:32:23

Django version 6.1.1, using settings 'CollegeOnline.settings'

Starting WSGI development server at **http://127.0.0.1:8000/**

Quit the server with CTRL-BREAK.




# Hello World

**Experiment 11:** Experiment Design HTML pages using Django templates with variables and template tags.

---

**1)Install Django and Create Project and App.**

```
PS C:\Users\Admin\Desktop\d_rutuja> python -m venv venv

PS C:\Users\Admin\Desktop\d_rutuja> .\venv\Scripts\activate

(venv) PS C:\Users\Admin\Desktop\d_rutuja> pip install django

Collecting django

  Using cached django-6.1.1-py3-none-any.whl.metadata (3.9 kB)

Collecting asgiref>=3.9.1 (from django)

  Using cached asgiref-3.12.1-py3-none-any.whl.metadata (9.4 kB)

Collecting sqlparse>=0.5.0 (from django)

  Using cached sqlparse-0.6.0-py3-none-any.whl.metadata (6.0 kB)

Collecting tzdata (from django)

  Using cached tzdata-2026.4-py2.py3-none-any.whl.metadata (1.4 kB)

Using cached django-6.1.1-py3-none-any.whl (8.4 MB)

Using cached asgiref-3.12.1-py3-none-any.whl (25 kB)

Using cached sqlparse-0.6.0-py3-none-any.whl (50 kB)

Using cached tzdata-2026.4-py2.py3-none-any.whl (347 kB)

Installing collected packages: tzdata, sqlparse, asgiref, django

Successfully installed asgiref-3.12.1 django-6.1.1 sqlparse-0.6.0 tzdata-2026.4

(venv) PS C:\Users\Admin\Desktop\d_rutuja> django-admin startproject myproject

(venv) PS C:\Users\Admin\Desktop\d_rutuja> cd myproject

(venv) PS C:\Users\Admin\Desktop\d_rutuja\myproject> python manage.py runserver
```

**2)Add App to setting.py**

```
INSTALLED_APPS = [
    'django.contrib.admin',
    'django.contrib.auth',
    'django.contrib.contenttypes',
    'django.contrib.sessions',
    'django.contrib.messages',
    'django.contrib.staticfiles',
    'myapp',
```

**3)Create templates folder**

**4)type program in index.html**

```
<!DOCTYPE html>
<html>
<head>
  <title>Django Template Example</title>
</head>
<body>
  <h1>Student Information</h1>
  <p>Name: {{ name }}</p>
  <p>Age: {{ age }}</p>
  {% if age >= 18 %}
    <p>Status: Eligible for Voting</p>
  {% else %}
    <p>Status: Not Eligible for Voting</p>
  {% endif %}
  <h2>Subjects</h2>
  <ul>
    {% for subject in subjects %}
      <li>{{ subject }}</li>
    {% endfor %}
  </ul>
</body>
</html>
```

**5)Add to View.py**

```python
from django.shortcuts import render

# Create your views here.
def home(request):
    data = {
        'name': 'Rutuja',
        'age': 20,
        'Subjects' : ['Python','Django','HTML','CSS']

    }
    return render(request,'index.html',data)
```

**6) Create urls.py in app**

```python
from django.urls import path
from . import views

urlpatterns = [
    path('', views.home, name='home'),
]
```

**7)Connect App URLs to project URLs**

Open myproject/urls.py

```python
from django.contrib import admin
from django.urls import path,include

urlpatterns = [
    path('admin/', admin.site.urls),
    path('',include('myapp.urls')),
]
```

**8) Run the Server**

In the terminal:

Python manage.py runserver

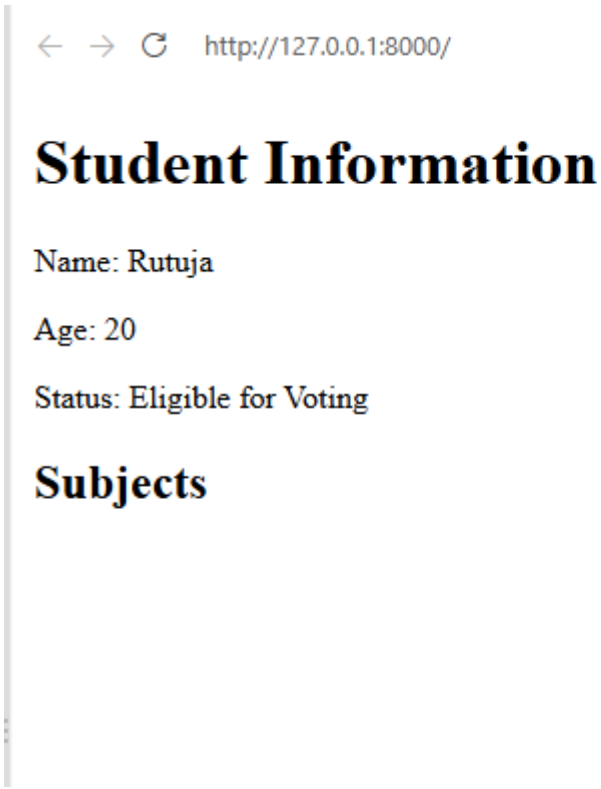
**9) You should see:**

Starting development server at http:/127.0.0.1:8000/

**10) open Browser**

at http:/127.0.0.1:8000/

**Output:**


http://127.0.0.1:8000/
Student Information
Name: Rutuja
Age: 20
Status: Eligible for Voting
Subjects